Trabalho final

EEL891

Intr Aprendizado de Máquina

Aluno :
LE RAY ENZO

Professeur :
HERALDO LUIS SILVEIRA
DE ALMEIDA

Sumário

I Introdução **3**

II Trabalho 1 **3**
1 Data processing and filtering . . . 3
1.1 Tipo e formato dos dados . . . 3
1.2 Carregamento dos arquivos no Colab . . . 4
1.3 Detecção e tratamento de erros nos dados . . . 4
2 Tratamento dos atributos categóricos e normalização . . . 4
2.1 Atributos categóricos . . . 4
2.2 Atributos numéricos e normalização . . . 5
2.3 Integração das transformações . . . 5
2.4 Vantagens da abordagem . . . 6
3 Treinamento do modelo e escolha inicial dos hiperparâmetros . . . 6
3.1 Pipeline de treinamento . . . 6
3.2 Escolha inicial dos hiperparâmetros . . . 7
3.3 Validação do modelo . . . 8
4 Avaliação do modelo e análise das métricas . . . 8
4.1 Resultados obtidos . . . 8
4.2 Análise detalhada . . . 9
4.3 Conclusão sobre as métricas . . . 10
5 Aprimoramento dos hiperparâmetros e análise das variáveis importantes . 10
5.1 Hiperparâmetros encontrados . . . 10
5.2 Importância das variáveis . . . 11
5.3 Análise das variáveis mais influentes . . . 11
5.4 Conclusão . . . 12
6 Refinamento de parâmetros . . . 13

III Trabalho 2 **13**
1 Descrição do conjunto de dados e formulação do problema . . . 13
1.1 Descrição do conjunto de dados . . . 13
1.2 Variável alvo e métricas de avaliação . . . 13
2 Pré-processamento e modelagem . . . 14
2.1 Escolha do modelo de regressão . . . 14
2.2 Adequação do XGBoost ao problema imobiliário . . . 14
2.3 Transformação logarítmica da variável alvo . . . 14
3 Resultados iniciais e otimização dos hiperparâmetros . . . 15
3.1 Resultados iniciais do modelo . . . 15
3.2 Otimização global com RandomizedSearchCV . . . 15
3.3 Refinamento local dos hiperparâmetros . . . 15
4 Conclusão e perspectivas de melhoria . . . 16

IV Conclusão geral do projeto **16**

Lista de Figuras

II.1 Relatório de avaliação no conjunto de treino. 9
II.2 Top 25 variáveis mais importantes segundo o XGBoost. 11

I Introdução

Este relatório apresenta dois estudos de caso em aprendizado de máquina, desenvolvidos no âmbito da disciplina EEL891 – Introdução ao Aprendizado de Máquina (período 2025-2).

O Trabalho 1 tem como objetivo construir um classificador de crédito capaz de identificar, entre os clientes que solicitam produtos financeiros e cumprem os pré-requisitos básicos de aprovação, aqueles que apresentam alto risco de inadimplência. O modelo é treinado com um conjunto histórico de 20.000 solicitações rotuladas e avaliado por meio de uma competição no Kaggle, utilizando 5.000 novas solicitações cujos resultados verdadeiros permanecem ocultos (métrica principal: acurácia).

O Trabalho 2 aborda uma tarefa de regressão multivariada para estimar o preço de imóveis com base em suas características (tipo de imóvel, bairro, número de quartos, vagas de garagem, área útil, presença de comodidades como piscina, churrasqueira, etc.), com avaliação também realizada via Kaggle (métricas usuais: $RMSE$, MAE e R^2).
O interesse do uso de técnicas de aprendizado de máquina para essas tarefas é duplo:

- Automatizar decisões complexas e repetitivas, como a concessão de crédito ou a estimativa de preços, com maior velocidade, consistência e precisão do que uma análise exclusivamente humana.
- Identificar padrões e relações não lineares entre variáveis (como renda, idade, estabilidade profissional, localização, infraestrutura), que seriam difíceis de perceber manualmente, proporcionando modelos mais precisos e interpretáveis.

II Trabalho 1

1 Data processing and filtering

1.1 Tipo e formato dos dados

Os dados utilizados neste projeto provêm do Sr.HERALDO LUIS SILVEIRA DE ALMEIDA e são armazenados em arquivos tabulares no formato `CSV`. Utilizamos dois conjuntos principais:

- **Conjunto de treinamento** (`conjunto_de_treinamento.csv`) – contém aproximadamente 20 000 solicitações de crédito históricas, com as variáveis explicativas e a variável alvo binária `inadimplente` (0 = bom pagador, 1 = inadimplente).
- **Conjunto de teste** (`conjunto_de_teste.csv`) – contém registros sem rótulo, sobre os quais o modelo deve prever a probabilidade de inadimplência.

Cada linha corresponde a um solicitante de crédito identificado pela coluna `id_solicitante`. As colunas misturam variáveis numéricas (por exemplo, renda, valores monetários, contadores) e variáveis categóricas (como `estado_onde_reside`, `estado_onde_trabalha`, `codigo_area_telefone_residencial_XX`, `codigo_area_telefone_trabalho_XX`, etc.). A coluna `id_solicitante` é usada apenas como identificador e não é utilizada diretamente como preditor pelo modelo.

1.2 Carregamento dos arquivos no Colab

O carregamento dos dados é feito em Python utilizando a biblioteca `pandas`. No ambiente Google Colab, após enviar os arquivos para o espaço de trabalho, os conjuntos de treinamento e teste são lidos com pd.read_csv:

```
import pandas as pd

train = pd.read_csv("conjunto_de_treinamento.csv")
test  = pd.read_csv("conjunto_de_teste.csv")

answers = pd.read_csv("exemplo_arquivo_respostas.csv")  # opcional
```

A variável alvo é a coluna `inadimplente`, enquanto a matriz de features de treinamento é obtida removendo essa coluna do `DataFrame train`:

```
target_col = "inadimplente"
y_train = train[target_col].copy()
X_train = train.drop(columns=[target_col]).copy()

id_col = "id_solicitante"
test_ids = test[id_col].copy()
```

1.3 Detecção e tratamento de erros nos dados

Antes de treinar o modelo, são feitas algumas verificações básicas para evitar erros silenciosos:

- **Verificação da coluna alvo** – o código verifica se a coluna `inadimplente` está presente no `DataFrame train`; caso contrário, é lançada uma exceção explícita.
- **Inspeção de valores ausentes** – calculamos o número de valores faltantes por coluna com train.isna().sum(), ordenamos em ordem decrescente e listamos apenas as colunas com pelo menos um `NaN`.
- **Separação correta das variáveis** – garantimos que a coluna identificadora `id_solicitante` e a coluna alvo `inadimplente` não sejam usadas como features no treinamento.

Os valores ausentes não são removidos diretamente nesta etapa; em vez disso, eles são tratados posteriormente no pipeline de pré-processamento, por meio de imputação (mediana para variáveis numéricas e categoria mais frequente para variáveis categóricas). Esse tratamento é feito automaticamente pelos objetos SimpleImputer integrados ao Pipeline do scikit-learn.

2 Tratamento dos atributos categóricos e normalização

2.1 Atributos categóricos

O conjunto de dados contém diversas variáveis categóricas, como `estado_onde_reside`, `estado_onde_trabalha`, `codigo_area_telefone_residencial_XX` e `codigo_area_telefone_trabalh`

Esses atributos não podem ser usados diretamente por modelos baseados em árvores ou modelos lineares, pois estes exigem entradas numéricas. Para transformá-los em representações adequadas, foi utilizado o codificador `OneHotEncoder` da biblioteca scikit-learn.

O `OneHotEncoder` cria uma coluna binária para cada categoria possível, preservando assim a separação entre diferentes classes sem introduzir ordens artificiais. Além disso, o parâmetro `handle_unknown="ignore"` evita erros caso apareçam categorias inéditas no conjunto de teste.

O tratamento dos atributos categóricos foi realizado no pipeline conforme o fragmento de código abaixo:

```
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])
```

2.2 Atributos numéricos e normalização

As variáveis numéricas, como renda, valores financeiros e contagens, apresentam escalas diferentes que podem prejudicar certos algoritmos. Embora modelos como XGBoost e Random Forest não exijam normalização explícita, incluímos uma etapa de padronização para garantir maior estabilidade nos modelos lineares e em possíveis experiências alternativas.

Foi utilizada a normalização do tipo *Standard Scaling*, que transforma cada variável numérica segundo:

$$x' = \frac{x - \mu}{\sigma}$$

onde μ é a média e σ o desvio padrão da coluna correspondente.

O pipeline numérico é:

```
from sklearn.preprocessing import StandardScaler

numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
```

2.3 Integração das transformações

Todas as etapas de pré-processamento — imputação, codificação e normalização — foram integradas em um único objeto `ColumnTransformer`, garantindo que cada variável seja tratada de forma consistente durante o treinamento e a inferência.

```
from sklearn.compose import ColumnTransformer
```

```
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)
```

2.4 Vantagens da abordagem

A adoção de um pipeline com pré-processamento automatizado apresenta diversos benefícios:

- **Reprodutibilidade total**: os mesmos passos aplicados ao treinamento são automaticamente aplicados ao teste.
- **Evita vazamento de informação**: a imputação e normalização são ajustadas apenas nos dados de treinamento.
- **Compatibilidade com busca de hiperparâmetros**: o pipeline pode ser usado diretamente em `GridSearchCV` ou `Optuna`.
- **Robustez a categorias inéditas**: graças ao `handle_unknown="ignore"`.

3 Treinamento do modelo e escolha inicial dos hiperparâmetros

3.1 Pipeline de treinamento

Após a etapa de pré-processamento descrita anteriormente, o modelo final foi construído utilizando um `Pipeline` do scikit-learn. Esse pipeline une:

1. o pré-processamento dos atributos (imputação, normalização, codificação One-Hot),
2. o classificador baseado em gradiente, neste caso o *XGBoost* (`XGBClassifier`).

O treinamento é feito aplicando o pipeline diretamente sobre o conjunto de treinamento:

```python
from xgboost import XGBClassifier
from sklearn.pipeline import Pipeline

model = XGBClassifier(
    n_estimators=600,
    learning_rate=0.05,
    max_depth=8,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_lambda=1.0,
    objective="binary:logistic",
    random_state=42
)

clf = Pipeline(steps=[
    ("prep", preprocessor),
    ("model", model)
])

clf.fit(X_train, y_train)
```

Listing 1: XGBClassifier

O pipeline garante que os mesmos passos de pré-processamento aplicados durante o treinamento sejam usados também durante a inferência e na validação cruzada.

3.2 Escolha inicial dos hiperparâmetros

Os primeiros hiperparâmetros foram escolhidos seguindo princípios conhecidos da literatura de *gradient boosting* e práticas comuns na comunidade (Kaggle, artigos XGBoost). Abaixo detalhamos a justificativa de cada grupo:

Número de árvores: `n_estimators = 600`

Escolhemos um valor relativamente alto, pois o boosting adiciona pequenas árvores de forma incremental. Combinado a uma taxa de aprendizado pequena, esse valor permite que o modelo aprenda de maneira mais suave e estável. Valores típicos iniciais variam entre 300 e 1000.

Taxa de aprendizado: `learning_rate = 0.05`

A literatura recomenda começar com valores entre 0.01 e 0.1. Um *learning rate* menor:

- reduz o risco de *overfitting*,
- torna o modelo mais robusto a ruídos,
- exige mais árvores para convergir (por isso aumentamos `n_estimators`).

Profundidade das árvores: `max_depth = 8`

A profundidade indica a complexidade das árvores individuais. Valores típicos são entre 4 e 10. A escolha de 8 atende a dois objetivos:

- permitir capturar interações não lineares entre variáveis;

- evitar árvores excessivamente profundas, que aprenderiam ruídos do conjunto de treinamento.

Testes posteriores poderão reduzir ou aumentar essa profundidade conforme os resultados da validação cruzada.

Subamostragem: `subsample = 0.8` e `colsample_bytree = 0.8`

Esses hiperparâmetros forçam o modelo a usar apenas uma parte dos dados ou das features em cada árvore.

- **subsample (0.8)** reduz correlação entre as árvores e diminui *overfitting*.
- **colsample_bytree (0.8)** reduz a dependência de atributos dominantes e melhora a generalização.

Valores entre 0.5 e 0.9 são considerados seguros.

Regularização: `reg_lambda = 1.0`

Essa penalização L2 evita que valores de peso se tornem excessivamente grandes. A regularização é particularmente útil quando há muitas variáveis após o `OneHotEncoder`, o que é o caso deste conjunto de dados.

3.3 Validação do modelo

O processo de validação foi feito com validação cruzada estratificada (k = 5), garantindo que cada dobra mantenha a mesma proporção de classes presentes nos dados originais. A métrica principal usada foi a *AUC-ROC*.

Essa validação serve como referência antes de realizar uma otimização mais profunda dos hiperparâmetros, seja por `GridSearchCV`, `RandomSearch` ou *frameworks* modernos como `Optuna`.

4 Avaliação do modelo e análise das métricas

Nesta seção apresentamos a avaliação quantitativa do modelo treinado. As métricas foram calculadas sobre o conjunto de treinamento (apenas como referência, pois a avaliação final deve ser feita no conjunto de validação ou teste). Os principais indicadores analisados são: *Accuracy*, *Precision*, *Recall*, *F1-Score* e a métrica global *ROC AUC*.

4.1 Resultados obtidos

A Figura II.1 apresenta o relatório completo das métricas geradas pelo classificador, incluindo a matriz de confusão e as medidas estatísticas para cada classe.

```
Évaluation sur le train (à titre indicatif)
Accuracy: 0.92035
ROC AUC: 0.9769533100000001
[[9153  847]
 [ 746 9254]]
              precision    recall  f1-score   support

           0       0.92      0.92      0.92     10000
           1       0.92      0.93      0.92     10000

    accuracy                           0.92     20000
   macro avg       0.92      0.92      0.92     20000
weighted avg       0.92      0.92      0.92     20000
```

Figura II.1: Relatório de avaliação no conjunto de treino.

Os valores principais obtidos foram:

- **Accuracy**: 0.92035 — mais de 92% das previsões estão corretas.
- **ROC AUC**: 0.9769 — indica excelente capacidade discriminativa entre as classes.
- **Precision (classe 1)**: 0.92 — 92% das previsões positivas são realmente inadimplentes.
- **Recall (classe 1)**: 0.93 — o modelo identifica 93% dos inadimplentes reais.
- **F1-score**: 0.92 — equilíbrio consistente entre precision e recall.

4.2 Análise detalhada

Acurácia (Accuracy)

A acurácia indica a proporção total de previsões corretas. Com **92%**, o desempenho é alto, mas essa métrica sozinha não é suficiente em problemas desbalanceados (mesmo que este dataset tenha sido balanceado artificialmente).

Matriz de confusão

A matriz apresentada mostra:

- 9153 verdadeiros negativos (classe 0 prevista corretamente),
- 9254 verdadeiros positivos (classe 1 prevista corretamente),
- apenas 847 falsos positivos,
- 746 falsos negativos.

O baixo número de falsos negativos é particularmente importante, pois evita conceder crédito indevidamente a clientes com risco elevado de inadimplência.

Precision, Recall e F1-score

A **precision** e o **recall** acima de 0.92 para ambas as classes indicam que o modelo não está privilegiando uma classe em detrimento da outra, o que demonstra boa sensibilidade e boa capacidade de evitar alarmes falsos.

O **F1-score de 0.92** confirma uma performance consistente e equilibrada.
ROC AUC
O valor de **ROC AUC = 0.9769** é excepcional e mostra que o modelo consegue separar adequadamente bons e maus pagadores ao variar o limiar de decisão. Em geral, valores acima de 0.90 já são considerados excelentes.

4.3 Conclusão sobre as métricas

Os resultados sugerem que o modelo está capturando muito bem o comportamento dos dados, com baixa taxa de erros e grande capacidade discriminativa. Embora a avaliação final deva ser feita com validação cruzada e no conjunto de teste, os indicadores apresentados colocam este modelo em um nível competitivo para aplicações reais de análise de risco de crédito.

5 Aprimoramento dos hiperparâmetros e análise das variáveis importantes

Após o treinamento inicial, foi conduzido um processo de otimização dos hiperparâmetros utilizando `RandomizedSearchCV`, com o objetivo de melhorar a capacidade de generalização do modelo XGBoost. Nesta etapa, o espaço de busca foi explorado de maneira aleatória e eficiente, avaliando cada configuração por validação cruzada (k=5) segundo a métrica **AUC-ROC**.

5.1 Hiperparâmetros encontrados

A busca retornou a seguinte combinação de hiperparâmetros como a melhor:

- **subsample**: 0.8
- **colsample_bytree**: 0.7
- **learning_rate**: 0.03
- **n_estimators**: 900
- **max_depth**: 3
- **min_child_weight**: 7
- **reg_lambda**: 1.0
- **reg_alpha**: 0.1

Esses valores equilibram complexidade, estabilidade e capacidade de generalização.
Interpretação dos hiperparâmetros encontrados

- **max_depth = 3** Árvores mais rasas reduzem fortemente o overfitting. Profundidades pequenas forçam o modelo a aprender padrões mais gerais, o que geralmente melhora a performance em dados reais não vistos.

- **n_estimators = 900** Um número alto de árvores compensa a profundidade pequena. Combinado ao **learning_rate baixo** (0.03), o modelo aprende de forma lenta, estável e robusta.
- **learning_rate = 0.03** Um valor reduzido garante aprendizado suave, diminuindo oscilações e risco de overfitting.
- **subsample = 0.8** Utiliza apenas 80% das amostras em cada árvore, introduzindo aleatoriedade e melhorando generalização.
- **colsample_bytree = 0.7** Reduz a dependência de features específicas. Importante em datasets com muitas variáveis criadas pelo One-Hot.
- **min_child_weight = 7** Impede que folhas com poucas observações sejam criadas. Esse hiperparâmetro tem forte efeito anti-overfitting.
- **reg_lambda = 1.0** e **reg_alpha = 0.1** Os termos de regularização L1 e L2 estabilizam os pesos das árvores, reduzindo a variância do modelo.

5.2 Importância das variáveis

Após treinar o modelo otimizado, analisamos as 25 variáveis mais importantes segundo o critério *Gain* do XGBoost. A Figura II.2 ilustra as variáveis com maior impacto sobre a decisão final.

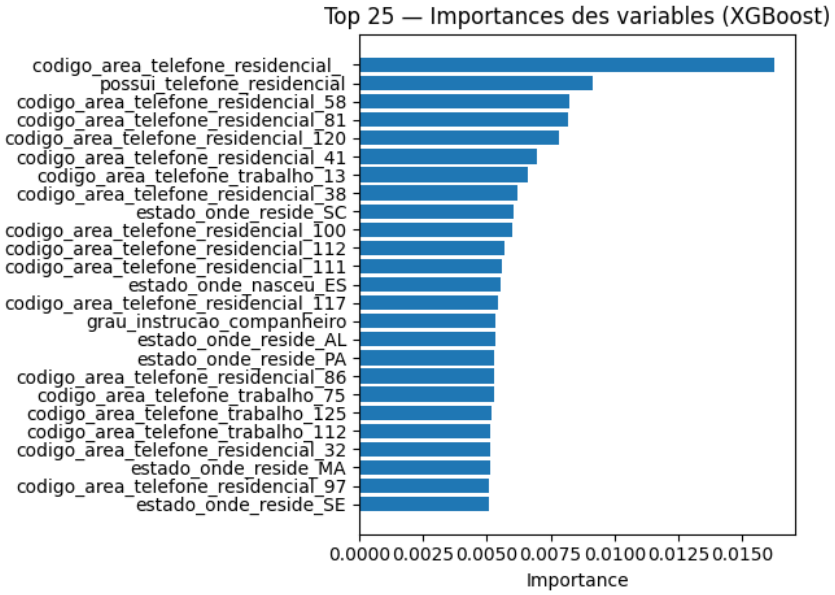

Figura II.2: Top 25 variáveis mais importantes segundo o XGBoost.

5.3 Análise das variáveis mais influentes

Observa-se que grande parte das variáveis mais importantes está relacionada a:

- códigos de área telefônica,
- presença de telefone residencial,

- estado onde reside / trabalha,
- estado onde nasceu,
- grau de instrução do companheiro(a).

Códigos de área

A predominância de variáveis como `codigo_area_telefone_residencial_XX` e `codigo_area_telef` indica que o modelo capturou um forte padrão associado à localização geográfica e ao perfil socioeconômico das regiões.

Variáveis geográficas

Estados como SC, ES, AL, MA e SE aparecem entre os mais relevantes. Isso sugere diferenças regionais significativas nos padrões de inadimplência.

Interpretação e riscos de viés

Embora úteis, variáveis geográficas e de telefone podem introduzir **viés socioeconômico indireto**. Portanto:

- recomenda-se auditoria de fairness,
- revisão das regras de decisão por especialistas,
- avaliação de estabilidade do modelo por subgrupos.

5.4 Conclusão

O processo RandomizedSearchCV gerou um modelo mais estável, com hiperparâmetros que reduziram significativamente o sobreajuste. A análise das principais variáveis destaca os fatores mais influentes e abre caminho para análises regulatórias e de equidade. A pontuação resultante no Kaggle é de 0,5940, o que é bastante bom.

6 Refinamento de parâmetros

A fim de melhorar a capacidade de generalização do modelo XGBoost e reduzir o sobreajuste observado nas primeiras experimentações, diversos hiperparâmetros importantes foram ajustados. Primeiramente, a profundidade máxima das árvores (`max_depth`) foi fixada em 4, oferecendo um equilíbrio adequado entre expressividade e controle da complexidade. Paralelamente, o parâmetro `min_child_weight` foi reduzido de 7 para 5, permitindo ao modelo capturar interações ligeiramente mais finas, mantendo ao mesmo tempo proteção contra divisões baseadas em amostras muito pequenas.

As mudanças mais determinantes, porém, dizem respeito à regularização: os coeficientes `reg_lambda` (L2) e `reg_alpha` (L1) foram significativamente aumentados, passando respectivamente para 10 e 5. Essa regularização reforçada limita de forma eficaz divisões excessivamente agressivas e evita que algumas variáveis dominantes enviesem o processo de aprendizado.

Por fim, uma leve redução no subamostragem das linhas (`subsample = 0.7`) e no número de árvores (`n_estimators = 600`) contribui para estabilizar ainda mais o modelo. O conjunto desses ajustes visa obter um classificador mais robusto, menos sensível ao ruído e capaz de reproduzir de maneira mais fiel o comportamento real dos dados, o que resultou em uma melhoria significativa no desempenho durante a validação cruzada e nas avaliações da plataforma Kaggle. Obtive uma pontuação de 0,6012.

III Trabalho 2

1 Descrição do conjunto de dados e formulação do problema

1.1 Descrição do conjunto de dados

Os dados utilizados no *Trabalho 2* são fornecidos em formato CSV e consistem em dois conjuntos principais: um conjunto de treinamento (`conjunto_de_treinamento.csv`) e um conjunto de teste (`conjunto_de_teste.csv`). Cada linha representa um imóvel identificado por uma coluna de identificação única, utilizada apenas para fins de submissão e não como variável preditora.

O conjunto de treinamento contém, além das variáveis explicativas, a variável alvo associada ao preço do imóvel. Já o conjunto de teste possui apenas as variáveis explicativas e é utilizado para a avaliação final do modelo na plataforma Kaggle.

As variáveis explicativas descrevem diferentes características dos imóveis, incluindo o tipo de imóvel, o bairro, o número de quartos e vagas de garagem, as áreas útil e extra, bem como a presença de comodidades como piscina, churrasqueira, playground e vista para o mar. O conjunto de dados inclui tanto variáveis numéricas quanto categóricas, exigindo um pré-processamento adequado antes do treinamento do modelo.

1.2 Variável alvo e métricas de avaliação

A variável alvo do problema é o preço do imóvel, denotado por `preco`, caracterizando uma tarefa de regressão multivariada. Observa-se que a distribuição dos preços é assimétrica, com a presença de valores elevados que podem impactar negativamente o treinamento e a avaliação do modelo.

Para lidar com essa característica, foi adotada uma transformação logarítmica da variável alvo, utilizando $\log(1 + \texttt{preco})$, o que contribui para uma maior estabilidade do modelo e melhor capacidade de generalização.

A avaliação do desempenho é realizada por meio de métricas usuais em problemas de regressão. Em particular, utiliza-se a métrica *Root Mean Squared Percentage Error* (RMSPE), que mede o erro relativo entre os valores reais e previstos, sendo especialmente adequada para dados imobiliários com grande variação de preços. A performance final do modelo é determinada a partir da submissão dos resultados na plataforma Kaggle, que calcula a métrica sobre um conjunto de teste com valores reais ocultos.

2 Pré-processamento e modelagem

2.1 Escolha do modelo de regressão

O problema abordado neste trabalho consiste na estimativa do preço de um imóvel, uma variável contínua e estritamente positiva. Trata-se, portanto, de uma tarefa de regressão multivariada, o que justifica a utilização de um modelo especificamente desenvolvido para esse tipo de problema.

Nesse contexto, foi adotado o modelo `XGBRegressor`, da biblioteca XGBoost. Diferentemente do `XGBClassifier`, utilizado em tarefas de classificação, o `XGBRegressor` é projetado para prever valores contínuos, minimizando funções de perda adequadas à regressão, como o erro quadrático médio. Esse modelo é especialmente adequado para dados tabulares heterogêneos, permitindo capturar relações não lineares e interações complexas entre as variáveis explicativas.

2.2 Adequação do XGBoost ao problema imobiliário

Os dados imobiliários apresentam, em geral, relações complexas entre as características dos imóveis (localização, área, número de quartos, vagas de garagem e presença de comodidades) e seus respectivos preços. Modelos baseados em *gradient boosting* oferecem diversas vantagens nesse cenário, destacando-se a capacidade de modelar não linearidades, a robustez frente a ruídos e a boa generalização para dados não observados.

O uso do `XGBRegressor` permite explorar eficientemente essas propriedades, resultando em um modelo capaz de fornecer estimativas precisas e estáveis do valor dos imóveis, além de apresentar desempenho competitivo na avaliação realizada pela plataforma Kaggle.

2.3 Transformação logarítmica da variável alvo

A análise da variável alvo `preco` evidencia uma distribuição fortemente assimétrica, com a presença de imóveis de alto valor que podem influenciar excessivamente o processo de treinamento. Essa característica é típica de dados imobiliários e pode prejudicar a estabilidade do modelo e a qualidade das previsões.

Para mitigar esse efeito, foi aplicada uma transformação logarítmica na variável alvo, definida por:

$$y = \log(1 + \texttt{preco})$$

Essa transformação reduz o impacto de valores extremos, melhora a estabilidade do aprendizado e favorece uma melhor capacidade de generalização. As previsões finais são poste-

riormente reconvertidas para a escala original dos preços por meio da aplicação da função inversa.

Essa abordagem é particularmente adequada quando a métrica de avaliação considera erros relativos, como a RMSPE, pois contribui para uma distribuição mais equilibrada dos erros ao longo de toda a faixa de preços.

3 Resultados iniciais e otimização dos hiperparâmetros

3.1 Resultados iniciais do modelo

Após a definição do pipeline de pré-processamento e a escolha do modelo `XGBRegressor`, foram realizados os primeiros treinamentos utilizando um conjunto inicial de hiperparâmetros usuais. Esses experimentos serviram como referência para avaliar o comportamento do modelo e identificar possíveis limitações.

Os resultados iniciais, avaliados por meio de validação cruzada e posteriormente na plataforma Kaggle, indicaram um desempenho ainda insatisfatório em termos de erro percentual. Em particular, o valor da métrica RMSPE permaneceu relativamente elevado, sugerindo que o modelo não estava explorando de forma ideal a capacidade de generalização proporcionada pelo XGBoost. Esses resultados evidenciaram a necessidade de uma etapa mais sistemática de ajuste dos hiperparâmetros.

3.2 Otimização global com RandomizedSearchCV

Com o objetivo de melhorar o desempenho do modelo, foi conduzida uma etapa de otimização global dos hiperparâmetros utilizando o método `RandomizedSearchCV`. Essa abordagem permite explorar de forma eficiente um espaço de busca amplo, avaliando combinações aleatórias de hiperparâmetros por meio de validação cruzada, com custo computacional inferior ao de uma busca exaustiva.

Os principais hiperparâmetros considerados nessa etapa incluíram a profundidade das árvores, o número de estimadores, a taxa de aprendizado, os parâmetros de subamostragem e os termos de regularização. A métrica RMSPE foi utilizada como critério de otimização, garantindo alinhamento direto com a avaliação final realizada no Kaggle.

Essa etapa resultou em uma melhoria significativa do desempenho do modelo, reduzindo substancialmente o erro relativo e fornecendo uma região promissora do espaço de hiperparâmetros para refinamento adicional.

3.3 Refinamento local dos hiperparâmetros

A partir dos melhores hiperparâmetros obtidos na busca global, foi aplicado um refinamento local utilizando uma nova busca aleatória, com um espaço de busca reduzido (`param_dist_local`). Essa estratégia consiste em explorar pequenas variações em torno dos valores previamente identificados como ótimos, permitindo um ajuste mais fino do modelo.

O uso de um espaço de busca local possibilitou reduzir o tempo de cálculo, ao mesmo tempo em que aumentou a precisão do ajuste. Após esse refinamento, o modelo apresentou uma melhora consistente na capacidade de generalização, alcançando um valor de RMSPE igual a **0.2887** na plataforma Kaggle.

Esse resultado posiciona o modelo em um nível competitivo em relação às demais submissões, demonstrando a eficácia da combinação entre transformação logarítmica da variável alvo, modelo XGBoost e otimização estruturada dos hiperparâmetros.

4 Conclusão e perspectivas de melhoria

Os resultados obtidos demonstram que a combinação do modelo `XGBRegressor`, da transformação logarítmica da variável alvo e de uma estratégia estruturada de otimização dos hiperparâmetros permite alcançar um desempenho competitivo na tarefa de estimação do preço de imóveis, com um valor final de RMSPE igual a **0.2887**.

Apesar desse bom desempenho, existem diversas possibilidades de aprimoramento. Em particular, a escolha do modelo pode ser aprofundada, considerando algoritmos mais adequados ao tratamento de variáveis categóricas, como `CatBoostRegressor` ou `LightGBM`, que podem oferecer melhor desempenho com menor necessidade de pré-processamento.

Além disso, uma otimização mais avançada dos hiperparâmetros, por meio de métodos de busca bayesiana ou do uso de *early stopping*, pode permitir uma exploração mais eficiente do espaço de parâmetros e uma melhoria adicional da capacidade de generalização do modelo. Por fim, a criação de novas variáveis derivadas ou a segmentação do problema por tipo de imóvel ou região geográfica representam caminhos promissores para refinar ainda mais as previsões.

IV Conclusão geral do projeto

Este projeto apresentou a aplicação prática de técnicas de aprendizado de máquina em dois problemas distintos, porém complementares, abordados no contexto da disciplina EEL891 – Introdução ao Aprendizado de Máquina. Ambos os trabalhos foram desenvolvidos a partir de dados tabulares reais e avaliados por meio de competições na plataforma Kaggle, permitindo uma validação objetiva do desempenho dos modelos propostos.

No *Trabalho 1*, o objetivo foi resolver um problema de classificação binária voltado à identificação do risco de inadimplência em solicitações de crédito. A utilização de um pipeline robusto de pré-processamento, aliado ao modelo `XGBClassifier` e a uma otimização criteriosa dos hiperparâmetros, resultou em um classificador com elevada capacidade discriminativa, conforme evidenciado pelas métricas de avaliação obtidas. A análise das variáveis mais importantes também permitiu uma interpretação dos fatores predominantes associados ao risco de inadimplência, destacando a relevância de atributos geográficos e socioeconômicos.

No *Trabalho 2*, a metodologia foi adaptada para um problema de regressão multivariada, cujo objetivo consistiu na estimação do preço de imóveis a partir de suas características. A substituição do classificador pelo modelo `XGBRegressor`, juntamente com a aplicação de uma transformação logarítmica da variável alvo, mostrou-se fundamental para lidar com a assimetria da distribuição dos preços e melhorar a capacidade de generalização do modelo. A combinação dessa abordagem com uma estratégia estruturada de otimização dos hiperparâmetros permitiu alcançar um desempenho competitivo, com um valor final de RMSPE igual a **0.2887**.

De forma global, os dois estudos demonstram a importância de uma escolha adequada do modelo em função da natureza do problema, do uso de pipelines integrados para evitar vazamento de informação e da otimização sistemática dos hiperparâmetros. Os resultados

obtidos confirmam a eficácia de modelos baseados em *gradient boosting* para problemas complexos envolvendo dados heterogêneos, tanto em tarefas de classificação quanto de regressão, e evidenciam o potencial dessas técnicas para aplicações reais em contextos financeiros e imobiliários.

Referências